
Temas (Opções) de Desenvolvimento – do Compilador MiniPar Orientado a Objetos 2026.1 com Real Paralelismo

**(olhar o Relatório de Compiladores-Helio – uso de canais de
comunicação)**

*(todas as opções de implementação, abaixo, serão aceitas com
a possibilidade da equipe correspondente de
obter a nota 10,0 na avaliação do projeto)*

**Objetivo: Fazer evoluir as duas implementações atuais do
MINIPAR 2025.1 com Real Paralelismo**

Implementação 1: Compilador da Linguagem MINIPAR não orientada a objetos, gerando código Assembly ARMV7 (roda OK no CPULator na Web) e código C e o executável.

Implementação 2: Interpretador da Linguagem Minipar Orientada a Objetos. Atualmente não gera código Assembly nem outro.

Para evoluir essas implementações são consideradas as seguintes opções de implementação:

Opção 1 – Compilador com Finalidade Didática – Implementação do Compilador da Linguagem MINIPAR 2026.1 Orientada a Objetos gerando o código Assembly ARMv7 para rodar na Web no CPULator.

Opção 2 (RUST) – Compilador com Finalidade de Melhoria da Performance da Execução – Implementação do Compilador da Linguagem MINIPAR 2026.1 Orientada a Objetos da forma seguinte:

- **Terá um interpretador** para a verificação sintática do programa Minipar Orientado a Objetos e não gerará código Assembly e **sim gerará código em RUST**, esse código RUST será compilado via o compilador da gnu, o gcc com a opção de otimização, posteriormente será gerado o **.exe** do programa Minipar em RUST e esse executável rodará mais rápido que a execução interpretada.

Opção 3 (C++) – Compilador com Finalidade de Melhoria da Performance da Execução – Implementação do Compilador da Linguagem MINIPAR 2026.1 Orientada a Objetos da forma seguinte:

- **Terá um interpretador** para a verificação sintática do programa Minipar e não gerará código Assembly e **sim gerará código em RUST**, esse código RUST será compilado via o compilador da gnu, o gcc com a opção de otimização, posteriormente será gerado o **.exe** do programa Minipar em C++ e esse executável rodará mais rápido que a execução interpretada.

Opção 4 (C) – Compilador com Finalidade de Melhoria da Performance da Execução – Implementação do Compilador da Linguagem MINIPAR 2026.1 Orientada a Objetos da forma seguinte:

- **Terá um interpretador** para a verificação sintática do programa Minipar e não gerará código Assembly e **sim gerará código em RUST**, esse código RUST será compilado via o compilador da gnu, o gcc com a opção de otimização, posteriormente será gerado o **.exe** do programa Minipar em C e esse executável rodará mais rápido que a execução interpretada.

Observação 1: Os relatórios das Implementações atuais da Linguagem MINIPAR serão disponibilizados para a Turma de Compiladores 2026.1

Observação 2: O compilador de um programa MiniPar Orientado a Objetos 2026.1 das opções 2, 3, e 4, será implementado via um Interpretador mais a geração de código equivalente em Rust ou C++ ou C. E posteriormente será gerado o **.exe** via o uso do compilador gcc (com a opção de otimização de código). **Essa geração do arquivo .exe pode ser automatizada via o uso de um script de comandos**

Observação 3: A Gramatica (BNF) a ser considerada para as 04 opções de implementação 2026.1 é aquela contida no Relatório do Interpretador da Linguagem Minipar Orientada a Objetos 2025.1 realizada pelos alunos:

Eduardo Maciel Alexandre
Josenilton Ferreira da Silva Junior
Lucas Cassiano Maciel dos Santos
Maria Letícia Ventura

Essa Gramatica (BNF) deverá ser completada para atender a todos os Programas de Teste 2026.1 e também os Testes Adicionais.

Testes adicionais (obrigatório)

Além dos Testes correspondentes para cada Tema (Opção), deve se implementar os dos testes seguintes:

Teste adicional 1 (obrigatório)

Programa de Execução de Real Paralelismo

- 1) o Algoritmo de Quick sort rodará no Computador 1
- 2) o Algoritmo de Multiplicação de duas Matrizes rodará no Computador 2
- 3) o Algoritmo de Fatorial de um número rodará no Computador 3

No computador 1 o programa Minipar 2026.1 deverá mostrar um Menu com as três opções de cálculo e via os canais de comunicação recolherá os resultados que serão apresentados na tela do computador 1.

Identificar cada Computador da forma seguinte: **IP, Porta**

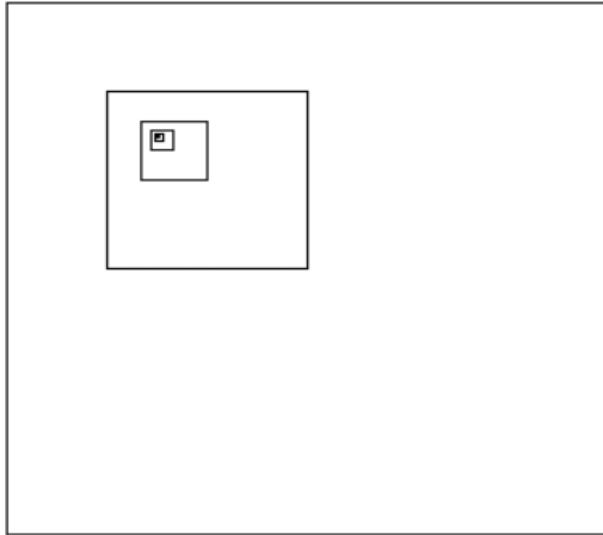
Realizar esse teste com 3 computadores no laboratório.

Teste adicional 2 (obrigatório)

Fractal

Desenha o fractal do tapete de Sierpinski recursivamente.

Exemplo da execução do código do Fractal:



Código do Fractal (Phyton):

```
import turtle

def draw_sierpinski_carpet(x, y, r, degree):
    """
    Desenha o fractal do tapete de Sierpinski recursivamente.
    x, y: coordenadas do centro do quadrado.
    r: metade do comprimento do lado do quadrado.
    degree: número de iterações.
    """
    if degree <= 0:
        return

    # Desenha o quadrado central que está sendo removido (ou o próprio quadrado)
    turtle.up()
    turtle.goto(x - r, y - r)
    turtle.down()
    # Usa uma cor para o "espaço" vazio, como branco, para simular a remoção
    # Ou uma cor de destaque para o centro preenchido, dependendo da interpretação do algoritmo.
    # O código original parece desenhar o quadrado, então vamos preencher o centro.
    turtle.fillcolor("white")
    turtle.begin_fill()
    for _ in range(4):
        turtle.forward(2 * r)
        turtle.left(90)
    turtle.end_fill()

    # Previne que o quadrado central seja desenhado nas chamadas recursivas,
    # caso o objetivo seja um tapete vazio no centro.
```

```

# Para o desenho da imagem, o quadrado central é desenhado.

# Chamadas recursivas para os 8 quadrados menores ao redor do centro
new_r = r / 3
new_degree = degree - 1

# Quadrante superior esquerdo (x-r, y+r) na definição da imagem
draw_sierpinski_carpet(x - new_r, y + new_r, new_r, new_degree)
# Quadrante superior direito (x+r, y+r) na definição da imagem
draw_sierpinski_carpet(x + new_r, y + new_r, new_r, new_degree)
# Quadrante inferior esquerdo (x-r, y-r) na definição da imagem
draw_sierpinski_carpet(x - new_r, y - new_r, new_r, new_degree)
# Quadrante inferior direito (x+r, y-r) na definição da imagem
draw_sierpinski_carpet(x + new_r, y - new_r, new_r, new_degree)

# Adicionando os outros 4 quadrados laterais/horizontais/verticais
draw_sierpinski_carpet(x - new_r, y, new_r, new_degree) # Esquerda
draw_sierpinski_carpet(x + new_r, y, new_r, new_degree) # Direita
draw_sierpinski_carpet(x, y + new_r, new_r, new_degree) # Cima
draw_sierpinski_carpet(x, y - new_r, new_r, new_degree) # Baixo

# O centro (x, y) não é chamado recursivamente.

def main():
    turtle.speed(0) # Velocidade máxima de desenho
    turtle.hideturtle()
    turtle.tracer(100, 0) # Acelera a visualização

    # Definir as dimensões iniciais
    initial_x = 0
    initial_y = 0
    initial_r = 150 # Metade do lado
    iterations = 60 # Número de iterações (grau de profundidade)

    # Iniciar o desenho do fractal
    # O primeiro quadrado é o maior, que contém todos os outros
    # Desenha o quadrado inicial grande (opcional, pode ser o fundo)
    # draw_square(initial_x, initial_y, initial_r)

    # Começa o processo recursivo que preenche os sub-quadrados
    draw_sierpinski_carpet(initial_x, initial_y, initial_r, iterations)

    turtle.update() # Atualiza a tela após desenhar tudo
    turtle.done()

if __name__ == "__main__":
    main()

```

Representar esse código do Fractal na linguagem Minipar Orientada a Objetos 2026.1

Simular a tela (área de desenho) via uma `matriz_Fractal`, e simular desenhar (na tela do computador) colocando na `matriz_Fractal` um carácter, pode ser um asterisco `*` ou um ponto `.` no local de desenho na `matriz_Fractal`:

`matriz_Fractal (i,j) = “.”`

Uma vez concluída a execução do código Minipar do Fractal desenhando na Matriz o Fractal correspondente então Mostrar na Tela do computador o resultado do Fractal gerado.